



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

ASSIGNMENT 5

Aadhya Maheshwari—1024150212

1.

Write a simple base class, then a derived class and use objects of both of them in the main function. It will be a simple illustration of inheritance.

main.cpp	Run	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 class Base { 5 public: 6 void showBase() { 7 cout << "This is Base Class" << endl; 8 } 9 }; 10 11 class Derived : public Base { 12 public: 13 void showDerived() { 14 cout << "This is Derived Class" << endl; 15 } 16 }; 17 18 int main() { 19 Base b; 20 Derived d; 21 22 b.showBase(); 23 d.showBase(); // inherited 24 d.showDerived(); 25 26 return 0; 27 }</pre>		<pre>This is Base Class This is Base Class This is Derived Class === Code Execution Successful ===</pre>

2.

Practice protected access specifier in inheritance. In the base class declare a variable which is protected and access it in the derived class.

main.cpp	Run	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 class Base { 5 protected: 6 int x; 7 public: 8 void setX(int a) { 9 x = a; 10 } 11 }; 12 13 class Derived : public Base { 14 public: 15 void display() { 16 cout << "Value of x: " << x << endl; 17 } 18 }; 19 20 int main() { 21 Derived d; 22 d.setX(10); 23 d.display(); 24 25 return 0; 26 }</pre>	Run	Value of x: 10 === Code Execution Successful ===

3.

Most of the time we use public mode of inheritance, for example:

```
class Derived : public Base {}
```

Try protected and private access modifiers to understand the difference of various modes of inheritance.

```
1 #include <iostream>
2 using namespace std;
3
4 class Base {
5 public:
6     int x;
7 };
8
9 class PublicDerived : public Base {
10 public:
11     void show() { cout << x << endl; }
12 };
13
14 class ProtectedDerived : protected Base {
15 public:
16     void set() { x = 20; cout << x << endl; }
17 };
18
19 class PrivateDerived : private Base {
20 public:
21     void set() { x = 30; cout << x << endl; }
22 };
```

```
24 int main() {
25     PublicDerived p;
26     p.x = 10;
27     p.show();
28
29     ProtectedDerived pr;
30     pr.set();
31
32     PrivateDerived pv;
33     pv.set();
34
35     return 0;
36 }
```

10
20
30

=== Code Execution Successful ===

4.

Various types of inheritances are as shown. Write small C++ codes for each inheritance type:

- Single inheritance

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 class A { 5 public: 6 void showA() { cout << "Class A" << endl; 7 } 8 }; 9 class B : public A { 10 public: 11 void showB() { cout << "Class B" << endl; 12 } 13 }; 14 int main() { 15 B obj; 16 obj.showA(); 17 obj.showB(); 18 }</pre>	<pre>Class A Class B === Code Execution Successful ===</pre>

- Multiple inheritance

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 class A { 5 public: 6 void showA() { cout << "A" << endl; } 7 }; 8 9 class B { 10 public: 11 void showB() { cout << "B" << endl; } 12 }; 13 14 class C : public A, public B { 15 }; 16 17 int main() { 18 C obj; 19 obj.showA(); 20 obj.showB(); 21 }</pre>	<pre>A B === Code Execution Successful ===</pre>

- Hierarchical inheritance

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 class A { 5 public: 6 void showA() { cout << "Base A" << endl; } 7 }; 8 9 class B : public A { 10 }; 11 12 class C : public A { 13 }; 14 15 int main() { 16 B b; 17 C c; 18 19 b.showA(); 20 c.showA(); 21 }</pre>	<pre>Base A Base A === Code Execution Successful ===</pre>

- Multilevel inheritance

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 class A { 5 public: 6 void showA() { cout << "A" << endl; } 7 }; 8 9 class B : public A { 10 }; 11 12 class C : public B { 13 }; 14 15 int main() { 16 C obj; 17 obj.showA(); 18 }</pre>	<pre>A === Code Execution Successful ===</pre>

- Hybrid inheritance

main.cpp	Run	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 class A { 5 public: 6 void showA() { cout << "A" << endl; } 7 }; 8 9 class B : public A { 10 }; 11 12 class C : public A { 13 }; 14 15 class D : public B, public C { 16 }; 17 18 int main() { 19 D obj; 20 obj.B::showA(); // avoid ambiguity 21 obj.C::showA(); 22 }</pre>		<pre>A A === Code Execution Successful ===</pre>

5. How can you use constructors and destructors in C++ inheritance? Write a program to illustrate.

main.cpp	Run	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 class Base { 5 public: 6 Base() { 7 cout << "Base Constructor" << endl; 8 } 9 ~Base() { 10 cout << "Base Destructor" << endl; 11 } 12 }; 13 14 class Derived : public Base { 15 public: 16 Derived() { 17 cout << "Derived Constructor" << endl; 18 } 19 ~Derived() { 20 cout << "Derived Destructor" << endl; 21 } 22 }; 23 24 int main() { 25 Derived d; 26 return 0; 27 }</pre>		<pre>Base Constructor Derived Constructor Derived Destructor Base Destructor === Code Execution Successful ===</pre>

6.

Implement a base class **Book** with attributes title, author, and price.

Create a derived class **Textbook** that adds subject.

Demonstrate how single inheritance is used to manage data.

```
1  #include <iostream>
2  using namespace std;
3
4  class Book {
5  protected:
6      string title, author;
7      float price;
8  public:
9      void input() {
10         cin >> title >> author >> price;
11     }
12 };
13 class Textbook : public Book {
14     string subject;
15 public:
16     void getData() {
17         input();
18         cin >> subject;
19     }
20     void display() {
21         cout << title << " " << author << " " <<
            price << " " << subject << endl;
22     }
23 };
24 int main() {
25     Textbook t;
26     t.getData();
27     t.display();
28 }
```

7.

A software company is creating a program to simulate a car's dashboard.

- Speed → Speedometer class
- Fuel level → FuelGauge class
- Temperature → Thermometer class

Create a **CarDashboard** class using multiple inheritance to display combined information.

main.cpp		Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 class Speedometer { 5 public: 6 int speed = 60; 7 }; 8 9 class FuelGauge { 10 public: 11 int fuel = 50; 12 }; 13 14 class Thermometer { 15 public: 16 int temp = 90; 17 }; 18 19 class CarDashboard : public Speedometer, public FuelGauge, public Thermometer { 20 public: 21 void display() { 22 cout << "Speed: " << speed << endl; 23 cout << "Fuel: " << fuel << endl; 24 cout << "Temperature: " << temp << endl; 25 } 26 }; 27 28 int main() { 29 CarDashboard c; 30 c.display(); 31 }</pre>	<pre>Speed: 60 Fuel: 50 Temperature: 90 === Code Execution Successful ===</pre>	

8.

Create a library system:

- Base class: **LibraryUser** (name, ID, contact)
- Derived classes: **Student** (grade level), **Teacher** (department)

Demonstrate hierarchical inheritance.

```
main.cpp  Run  Output  Clear

1  #include <iostream>
2  using namespace std;
3
4  class LibraryUser {
5  protected:
6      string name;
7  public:
8      void input() { cin >> name; }
9  };
10
11 class Student : public LibraryUser {
12     int grade;
13 public:
14     void get() {
15         input();
16         cin >> grade;
17     }
18     void show() {
19         cout << name << " " << grade << endl;
20     }
21 };
22
23 class Teacher : public LibraryUser {
24     string dept;
25 public:
26     void get() {
27         input();
28         cin >> dept;
29     }
30     void show() {
31         cout << name << " " << dept << endl;
32     }
33 };
34
35 int main() {
36     Student s;
37     Teacher t;
38
39     s.get();
40     t.get();
41
42     s.show();
43     t.show();
44 }
```

Aadhya
10
Manya
9
Aadhya 10
Manya 9

=== Code Execution Successful ===

9.

Vehicle management system:

- Base class: **Vehicle** (make, model, year)
- Derived: **Truck** (load_capacity)
- Further derived: **RefrigeratedTruck** (temperature_control)

Demonstrate multilevel inheritance.

```
1 #include <iostream>
2 using namespace std;
3
4 class Vehicle {
5 protected:
6     string make;
7 public:
8     void input() { cin >> make; }
9 };
10
11 class Truck : public Vehicle {
12 protected:
13     int load;
14 public:
15     void get() {
16         input();
17         cin >> load;
18     }
19 };
20
21 class RefrigeratedTruck : public Truck {
22     int temp;
23 public:
24     void getData() {
25         get();
26         cin >> temp;
27     }
28     void show() {
29         cout << make << " " << load << " " << temp <<
30             endl;
31     };
32
33 int main() {
34     RefrigeratedTruck r;
35     r.getData();
36     r.show();
37 }
```

Car
100
5000
Car 100 5000

=== Code Execution Successful ===

10.

Academic system:

Base class: **Person** (name, address)

Derived: **Staff** (employee_id, department)

Derived: **Student** (student_id, grade)

Hybrid: **TeachingAssistant** inherits from both Staff and Student

Demonstrate hybrid inheritance.

```
1  #include <iostream>
2  using namespace std;
3
4  class Person {
5  protected:
6      string name;
7  public:
8      void input() { cin >> name; }
9  };
10
11 class Staff : public Person {
12 protected:
13     int emp_id;
14 public:
15     void getStaff() {
16         input();
17         cin >> emp_id;
18     }
19 };
20
21 class Student : public Person {
22 protected:
23     int student_id;
24 public:
25     void getStudent() {
26         input();
27         cin >> student_id;
28     }
29 };
30
31 class TeachingAssistant : public Staff, public
    Student {
32 public:
33     void display() {
34         cout << "TA Info:" << endl;
35         cout << "Staff ID: " << emp_id << endl;
36         cout << "Student ID: " << student_id <<
            endl;
37     }
38 };
39
40 int main() {
41     TeachingAssistant ta;
42
43     ta.getStaff();
44     ta.getStudent();
45     ta.display();
46
47     return 0;
48 }
```

212
100
100
212
TA Info:
Staff ID: 100
Student ID: 212

=== Code Execution Successful ===